Exercise Sheet 06

The Full Moon

House rules.

- One file per exercise, in `my_work/`: `ex1_traceback.py`, `ex2_try_except.py`, …
- Run everything **from the lesson folder**. The broken file is `data/quests_broken.csv`; leave it broken.
- Today, errors are the point. When something crashes, read the **last line** first (what), then the **line number** (where), then decide: fix the code, or catch the error on purpose?
- Hand in: commit and push to `lesson-06`, open a pull request, and tell Lars which exception type you met most often.

Exercise 1 Read the traceback

8 min · `ex1_traceback.py`

> **RAVEN #601, from Orrin of the Clockworks:** *"Before you catch errors, learn to read them. Python tells you exactly what went wrong. People just don't look. Dalvric didn't look for forty years."*

GIVEN (TYPE IT EXACTLY, INCLUDING THE MISTAKE)

```
heroes = ["Bram", "Wren", "Piran"]
levels = [11, 19, 4]
for i in range(4):
    print(heroes[i], levels[i])
```

(a) Run it. It crashes. Write the **line number** and the **error type** as a comment in the file. Then fix the loop so it works for any number of heroes.

(b) Add `print(level)` at the bottom (singular, on purpose). Run it, write down the error type, then replace it with a line that prints the party's total level.

EXPECTED OUTPUT

```
Bram 11
Wren 19
Piran 4
Party level: 34
```

Hint: A traceback reads bottom-up: the last line is the error, the line above it points at your code. The error types you'll meet today: `IndexError`, `NameError`, `ValueError`, `TypeError`, `ZeroDivisionError`, `KeyError`.

Exercise 2 Catch it

10 min · `ex2_try_except.py`

> **RAVEN #602, from Quartermistress Pell:** *"The Scriptorium writes 'n/a' in the reward column when the Queen hasn't decided yet. Your lovely script from last moon falls over on row 2. Can it just… carry on? P."*

GIVEN

```
raw_values = ["1,240 cr", "n/a", "", "610", "lots", "2,850.50 cr"]
```

(a) Loop over the values. Remove "cr", commas and spaces, then try to turn the text into a float **inside a `try` block**.

(b) If it works, print the original text (with `!r`, left-aligned in 14) and the number. If it raises a `ValueError`, print "could not read" instead.

(c) Count the good and the bad ones and print the totals.

EXPECTED OUTPUT

```
'1,240 cr'     -> 1240.0
'n/a'          -> could not read
''             -> could not read
'610'          -> 610.0
'lots'         -> could not read
'2,850.50 cr'  -> 2850.5
Read 3, skipped 3
```

Hint: `f"{text!r:<14}"` shows the quotes and pads to 14. Put the "it worked" print in an `else:` after the `except:`, so only the risky line sits inside `try`.

Exercise 3 Which error is it?

10 min · `ex3_which_error.py`

> **RAVEN #603, from Captain Siv, quest master:** *"'Something went wrong' is not a reason. Zero days on the road is a different problem from a missing reward. Tell me which. One word will do."*

GIVEN

```
pairs = [(1320, 9), (950, 0), ("tbc", 6), (480, None), (3420, "16")]
```

(a) Each pair is (reward, days). For each pair, compute the crowns per day: `float(reward) / float(days)`, inside a `try`.

(b) Catch `ZeroDivisionError`, `ValueError` and `TypeError` in three separate `except` blocks, each with its own message.

(c) In the `else` block, print the crowns per day with 2 decimals.

EXPECTED OUTPUT

```
1320, 9: 146.67 cr per day
950, 0: zero days, did they even leave?
tbc, 6: that's not a number
480, None: a value is missing
3420, 16: 213.75 cr per day
```

Hint: Before you run it, write next to each pair which error you expect. `float("16")` is fine; `float(None)` is not.

Exercise 4 Raise your own

10 min · `ex4_raise.py`

> **RAVEN #604, from Captain Siv:** *"Difficulty goes from 1 (giant rats) to 5 (wyrm). Somebody pinned up a 7 and the Grimoire sent two Bards and a lute. I want the script to refuse, loudly."*

GIVEN

```
difficulties = [3, 7, 0, "4", 5]
```

(a) Write `check_difficulty(difficulty)`. If it is not an `int`, `raise TypeError` with a message that includes the value (use `!r`). If it's outside 1–5, `raise ValueError` with a message that includes the number. Otherwise return it.

(b) Loop over the difficulties and call it inside `try`. Print "Rejected:" and the message for each error, "OK:" and the difficulty otherwise.

(c) Add a `finally:` block that counts how many notices were checked, and print the count at the end.

EXPECTED OUTPUT

```
OK: 3
Rejected: difficulty must be between 1 and 5, got 7
Rejected: difficulty must be between 1 and 5, got 0
Rejected: difficulty must be a whole number, got '4'
OK: 5
Checked 5 notices
```

Hint: `except ValueError as e:` puts the message in `e`. `isinstance(difficulty, int)` asks "is this an int?". `finally` runs every time, error or not, which is exactly what a counter wants.

Exercise 5 The broken file

12 min · `ex5_broken_file.py`

> **RAVEN #605, from Quartermistress Pell:** *"`data/quests_broken.csv` is this moon's copy of the ledger. Four hundred quests, and I know at least four are wrong. Find them, skip them, and give me the average reward per day on the road for everything else."*

GIVEN

```
import csv
```

(a) Read the file with `csv.DictReader`. For each row, inside one `try`: compute the reward per day (reward / days, converting with `float()`) and the difficulty as an `int()`.

(b) Catch `ValueError`, `TypeError` and `ZeroDivisionError` together in one `except` as `e`. Store the quest's id and `type(e).__name__` for each bad row; store the reward per day for each good one.

(c) Print the counts, the bad rows with their error type, and the average reward per day of the good rows.

EXPECTED OUTPUT

```
Good rows: 394
Bad rows: 6
  Q0005  ValueError
  Q0018  ValueError
  Q0034  ZeroDivisionError
  Q0059  ValueError
  Q0092  ValueError
  Q0121  TypeError
Average reward per day: 74.25 cr
```

Hint: Catch several types at once with a tuple: `except (ValueError, TypeError, ZeroDivisionError) as e:`. One row is missing its last six columns, so `row["reward"]` is `None`: that's the TypeError.

Exercise 6 Print debugging

8 min · `ex6_print_debugging.py`

> **RAVEN #606, from Queen Aldara, at dusk:** *"The loot pool script says 0 crowns. Wren says that can't be right, and Wren is rarely wrong about crowns. I would like it to be right, but I'd settle for knowing why it isn't. A."*

GIVEN (TYPE IT EXACTLY; IT HAS A LOGIC BUG, NO CRASH)

```
loot = {"Bram": 1240, "Wren": 2480, "Piran": 310, "Biscuit": 0}
for name, share in loot.items():
    pool = 0
    if share > 0:
        pool += share
print("Loot pool: " + f"{pool:,}" + " cr")
```

(a) Run it. Then add a `print(name, share, pool)` inside the loop, run again, and *watch* the pool. Where does the loot go?

(b) Fix the bug. Remove your debugging print. Add a second loop that counts how many heroes got a share, and print it. (Biscuit came along. Biscuit got nothing.)

EXPECTED OUTPUT

```
Loot pool: 4,030 cr
Heroes with a share: 3
```

Hint: Printing a variable inside a loop is the oldest debugging tool there is, and it still works. VS Code's debugger (lecture) does the same thing without editing the file: put a breakpoint on `pool += share`, press F5, step with F10.

Exercise 7 A safety net STRETCH

12 min · `ex7_safe_float.py`

> **RAVEN #607, from Orrin:** *"You'll write 'try float, except give up' a hundred more times. Write it once, as a function, and make it log what it gave up on. Dalvric used four hundred lines. Please use fewer."*

GIVEN

```
import csv
```

(a) Write `safe_float(text, default=None)`: returns `float(text)`, or the default if that raises `ValueError` or `TypeError`.

(b) Read the broken file. For every row and each of the columns `difficulty`, `days`, `reward`: if `safe_float` gives `None`, count it per column in a dict and add a line "Qxxxx: column = 'value'" to a list.

(c) Print the counts per column, write the lines to `my_work/bad_rows.txt`, print how many you logged, then print the file.

EXPECTED OUTPUT

```
reward: 4 unreadable
difficulty: 1 unreadable
days: 1 unreadable
Logged 6 problems to bad_rows.txt
Q0005: reward = 'n/a'
Q0018: reward = ''
Q0059: difficulty = 'deadly'
Q0092: reward = 'tbc'
Q0121: days = None
Q0121: reward = None
```

Hint: Zero days is a valid float, so Q0034 doesn't appear here. That's the difference between "unreadable" and "wrong": a safety net catches the first kind. The second kind needs a human, or a `raise`.

Bug hunt Pages from the Grimoire, page 6

15 min · `bughunt/robust_bug.py`

Dalvric's "robust" total of every reward on the ledger, with warding spells he added after the Scriptorium's copy broke one full moon. There are **three** bugs: one crashes inside the error handling itself (a misspelt ward: read *both* tracebacks), one double-counts crowns without anyone noticing, and one is off by exactly one.

EXPECTED OUTPUT ONCE FIXED

```
Total of all rewards: 233,490 cr
Skipped 4 rows: Q0005, Q0018, Q0092, Q0121
```

Commit each fix separately. For the double-counting bug, explain in the commit message *which* quest's reward was counted twice.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.